

Economy RP.

Equipo de Trabajo Grupo 4.

Asesor

Juan Camilo Palacio Alcaraz

Integrantes

Miguel Ángel Cardona Agudelo

Luisa María López Orrego

Jorge Luis Ordoñez Ávila



--	--	--

TABLA DE CONTENIDO

	Pág.
1. Descripción del negocio	2
2. Objetivos del negocio.....	3
3. VISTA EXTERNA	3
3.1 ACTORES Y ROLES	4
3.2 MODELO VERBAL.....	4
4. REGLAS DE NEGOCIOS.....	5
5. DESCRIPCIÓN DEL PROBLEMA.....	5
6. MODELO CONCEPTUAL DE LA SOLUCIÓN.....	5
0.1 Modelado piloto (Primera versión)	5
0.2 Modelado 2.0 (Orden cascada).....	5
0.3 Modelado final (Estructura lineal)	5
7. MODELO LÓGICO DE LA SOLUCIÓN.....	5
1FN: Eliminar atributos multivaluados o repetidos.	5
2FN: Eliminar dependencias parciales.	5
3FN: Eliminar dependencias transitivas.....	5

1. Descripción del negocio

Los servidores privados de rol con temática GTA (montados sobre plataformas como FiveM, RedM, RAGE MP, Grand RP) sostienen una economía interna donde cada jugador gana dinero a través de empleos simulados dentro del juego y con ese dinero adquiere vehículos, propiedades, negocios y otros bienes que puede usar, mejorar o revender a otros jugadores. Actualmente esta economía se administra mediante scripts sueltos que cada servidor instala por separado, sin un sistema central que registre de forma ordenada quién es dueño de qué, de dónde salió ese dinero o ese bien, y qué transacciones se han hecho entre los jugadores. Cuando dos jugadores hacen un intercambio o una venta, el administrador del servidor no tiene forma sencilla de verificar el historial de esa transacción ni de detectar si un jugador obtuvo un bien por fuera de las reglas del servidor, lo que abre espacio a fraudes, duplicación de ítems y pérdida de confianza en la economía del juego. El negocio que se propone es un motor de gestión económica que centraliza el registro de jugadores, empleos, bienes y transacciones en una base de datos relacional, de modo que cualquier servidor de rol pueda conectarse a él y tener trazabilidad completa sobre su economía interna.

2. Objetivos del negocio

El objetivo central del negocio es que el tradeo entre jugadores deje de ser una apuesta de confianza y se convierta en una operación segura y verificable. Se busca frenar la hiperinflación y el desbalance que hoy generan los tradeos sin control, donde un jugador puede aparecer con bienes de la nada o duplicar lo que ya tiene. También se busca proteger la confianza del mercado de intercambios, de manera que cualquier jugador pueda tradear sabiendo que la otra parte no le va a

incumplir a mitad de la operación. Otro objetivo es erradicar el comercio ilícito de ítems dentro del tradeo, es decir, bienes duplicados o de origen desconocido que terminan circulando entre jugadores y devaluando el resto de la economía. Por último, se busca reducir las pérdidas económicas que hoy sufren los jugadores cuando un tradeo queda a medias, protegiendo tanto el bien como el dinero de cada uno de los dos jugadores involucrados.

A demás de minimizar las mermas derivadas de desbalances salariales y disminuir el impacto financiero provocado por las extensas horas de soporte técnico dedicadas a resolver quejas producto de transacciones informales fallidas.

3. VISTA EXTERNA

3.1 ACTORES Y ROLES

-Personas involucradas en el negocio

Jugador: usuario del servidor de rol que participa en el tradeo, administra su cuenta y su inventario de bienes, y puede ofrecer, aceptar o rechazar propuestas de intercambio con otros jugadores.

Jugador comerciante: jugador que dentro de la trama del servidor vende vehículos o propiedades a otros jugadores, manejando un concesionario o un negocio de bienes raíces dentro del rol, y que le da a los jugadores buena parte de los bienes que después terminan tradeando entre ellos.

Moderador de economía: encargado de revisar los reportes de jugadores sobre tradeos con problemas, como bienes con restricciones ocultas o intentos de fraude durante la negociación, y de escalar los casos graves al administrador.

Administrador del servidor: encargado de configurar los parámetros económicos del servidor, como salarios, precios base o límites de bienes por jugador, de

intervenir en casos de fraude reportado en un tradeo y de consultar el historial de transacciones cuando surja una disputa entre jugadores.

3.2 MODELO VERBAL

En la dinámica económica actual del servidor, los jugadores obtienen su capital inicial desempeñando empleos simulados dentro del juego o adquiriendo bienes primarios a través de un comerciante. Sin embargo, el verdadero volumen financiero y la circulación de los activos de mayor valor, como vehículos de gama alta y bienes raíces, se concentra en el comercio e intercambio directo entre jugadores. Hoy en día, este proceso se ejecuta completamente a ciegas, pues las partes se contactan dentro del entorno de rol, acuerdan de palabra la transacción, ya sea una permuta de activos, un pago en efectivo o una combinación de ambos, y proceden a realizar el traspaso confiando únicamente en la buena fe de la contraparte. En esta instancia no existe ningún tipo de validación previa ni en tiempo real por parte del sistema, ya que no se verifica si el bien ofrecido pertenece realmente al inventario del emisor, no se evalúa si el activo carga con deudas o embargos pendientes, ni se comprueba si el comprador dispone del saldo suficiente antes de ejecutar la entrega. Al carecer de mecanismos de control automático y bloqueos de seguridad, el proceso de intercambio queda totalmente expuesto a fallas operativas y conductas malintencionadas. Es común que un jugador ofrezca el mismo vehículo en dos negociaciones distintas de forma simultánea, que la transacción se interrumpa abruptamente a mitad de camino por una desconexión de red intencionada o accidental dejando los activos en un limbo, o que uno de los participantes altere los términos acordados a último segundo cuando la otra parte ya daba por cerrado el trato. Como no existe un mecanismo transaccional que unifique la operación, el intercambio se rompe y ninguno de los usuarios recibe un respaldo o comprobante claro de lo que acaba de suceder en la capa de datos. Esta vulnerabilidad traslada el problema directamente a la gestión administrativa. Cuando un jugador afectado radica un reporte por estafa o cuando la comunidad detecta la aparición de un bien duplicado, la resolución de la disputa

se convierte en un calvario manual, lento y deficiente. El administrador o moderador se ve obligado a reconstruir los hechos consultando registros completamente dispersos e incompletos, tales como archivos de texto de consola generados por scripts independientes que no se comunican entre sí, bases de datos locales sin relación de integridad, e incluso evidencias informales como capturas de pantalla o videos aportados por los propios usuarios en canales externos. Al no contar con una línea de tiempo unificada ni con registros inmutables, la investigación consume horas de trabajo operativo y con frecuencia resulta imposible determinar con certeza qué fue lo que verdaderamente acordaron las partes. En consecuencia, las quejas se terminan resolviendo al arbitrio o criterio personal del administrador y no sobre un historial técnico verificable, lo que desgasta al equipo de soporte y alimenta un ambiente de profunda desconfianza en la estabilidad económica del servidor

4. REGLAS DE NEGOCIOS

CODIGO	NOMBRE	DESCIPCION	FORMULA	FUENTE	REGLAS RELACIONADAS
RN001	Registro de único laboral	Cuando un jugador completa un empleo dentro del servidor, el sistema debe calcular y acreditar automáticamente la ganancia correspondiente a su cuenta, dejando registro de la fecha y	Ganancia = Tarifa_base_empleo x Horas_trabajadas	Equipo de desarrollo o.	RN003 RN005

		el empleo que la generó.			
RN002	Propiedad Única y Exclusiva	Un bien físico específico (un vehículo con una placa determinada, una casa en una dirección exacta) solo puede pertenecer a un (1) personaje a la vez	(Propietario) = 1 para cada \$Bien_id activo	Lógica de bienes raíces y vehículos	RN003 RN004
RN003	Inmutabilidad del Historial	Una vez que una transacción (compra, venta, transferencia, pago de salario) es registrada en el sistema, no puede ser modificada, editada ni eliminada bajo ninguna circunstancia.	Política de base de datos	Auditoría y Seguridad	RN001, RN002, RN004, RN005
RN004	Restricción de transferencia por deuda	Un bien no puede transferirse ni revenderse a otro jugador si tiene una deuda o una cuota pendiente asociada, como un vehículo	N/A	Equipo de desarrollo	RN002, RN003

		financiado que aún no ha sido pagado en su totalidad.			
RN05	Trazabilidad de Creación de Dinero	Todo dinero inyectado a la economía que no provenga de otro jugador (ej. pago de salarios, bonos iniciales) debe tener como cuenta de origen la cuenta oficial del "Sistema" del servidor.	Origen = "Cuenta_Sistema" si Emisor = Personaje	Equipo de desarrollo	RN001, RN003
RN006	Comisión por Transferencia	Toda compraventa o intercambio entre jugadores debe descontar un porcentaje fijo del monto transferido como comisión, que se acredita a la cuenta del "Sistema" y no a ningún jugador, funcionando como un mecanismo de drenaje para frenar	Comisión = Monto_transaccion x Porcentaje_comision	Equipo de desarrollo	RN001, RN005

		la hiperinflación del servidor.			
RN007	Límite Máximo de Propiedades	Un jugador no puede tener registrados a su nombre más vehículos o propiedades del límite que defina el administrador del servidor, para evitar que un solo jugador acapare bienes y desbalancee la oferta disponible para el resto de la comunidad.	Cantidad_bienes(Jugador) ≤ Limite_definido	Administrador del servidor	RN002
RN008	Periodo de Enfriamiento Laboral	Un jugador debe esperar un tiempo mínimo definido por el servidor entre el final de una jornada laboral y el inicio de la siguiente en ese mismo empleo, para que no pueda trabajar de forma ininterrumpida y generar ingresos	Hora_actual - Hora_ultima_jornada ≥ Tiempo_enfriamiento	Equipo de desarrollo	RN001

		desproporcionados frente al resto de jugadores.			
RN009	Penalización por Bancarrota	Si el saldo de un jugador queda en negativo o no alcanza a cubrir sus deudas pendientes, el sistema lo declara en bancarrota, congela temporalmente su capacidad de realizar transacciones y le aplica una penalización económica definida por el administrador antes de dejarlo operar con normalidad otra vez.	N/A	Administrador del servidor	RN003, RN009
		Todo tradeo debe ser confirmado explícitamente por los dos jugadores involucrados sobre los términos finales (bien y dinero			

RN010	Confirmación Doble en el Tradeo	ofrecidos) antes de que el sistema ejecute la transferencia. Si alguno modifica los términos después de que el otro ya confirmó, la confirmación se reinicia para evitar cambios de última hora.	N/A	Equipo de desarrollo	RN003, RN013
RN011	Límite Diario de Tradeos	Un jugador no puede ejecutar más de 5 tradeos dentro de un mismo día, buscando evitar que las mismas cuentas laven bienes o dinero mediante tradeos masivos y repetitivos entre ellas.	Tradeos del día(Jugador) <= 5	Administrador del servidor	RN006
		Un jugador solo puede tener abierta una negociación de tradeo a la vez. Mientras esa negociación esté			

RN012	Tradeo Único Simultáneo	pendiente de confirmación no puede iniciar ni aceptar ninguna otra propuesta, lo que evita que el mismo bien o el mismo saldo se comprometa en más de una operación al mismo tiempo.	N/A	Equipo de desarrollo	RN002, RN011
--------------	-------------------------------	--	-----	----------------------	--------------

5. DESCRIPCIÓN DEL PROBLEMA

5.1 Características generales de la solución: La solución debe garantizar que ningún intercambio quede a medias, de forma que un jugador nunca pierda un bien sin recibir lo acordado a cambio. Debe ser verificable, permitiendo confirmar en cualquier momento que un bien que un jugador dice tener realmente le pertenece dentro del sistema. Debe ser auditable, dejando un historial completo de cada intercambio para que el administrador pueda revisarlo ante una disputa. Y debe funcionar de forma independiente del resto de la economía del servidor, es decir, sin necesitar saber cómo un jugador obtuvo originalmente un bien, sino únicamente validando su estado actual en el inventario.

5.2 Causas de problema a aliviar:

- **Ejecución no atómica de transferencias P2P:** Transferir dinero y propiedad consultas independientes sin control de transacciones provoca duplicaciones o pérdidas de bienes si ocurre un fallo de red o una desconexión a mitad del proceso.
- **Falta de verificación de saldo e inventarios en tiempo real:** Procesar compras sin consultas bloqueantes ni comprobación inmediata de fondos

genera saldos negativos, cobros dobles y entregas de bienes a costo cero por desincronización.

- **Generación de dinero ficticio sin validación:** Los empleos acreditan saldo desde el lado del cliente sin verificar en la base de datos, lo que facilita la modificación de valores en memoria, cobros de jornadas falsas y duplicación de pagos sin trazabilidad.
- **Falta de depreciación y restricciones de transferencia:** No evaluar el uso ni revisar deudas o embargos antes de una reventa permite transferir bienes gravados o sobrevalorados, trasladando el problema financiero al comprador.
- **Inexistencia de un historial centralizado:** La ausencia de una bitácora de transacciones impide a los administradores investigar disputas entre usuarios, auditar movimientos sospechosos y ajustar parámetros para controlar la inflación del servidor.

5.3 Breve descripción de los procesos:

Proceso de tradeo entre jugadores: en la situación actual, cuando un tradeo falla a mitad de camino por una desconexión o porque uno de los dos cambia lo acordado de última hora, no solo se pierde el bien o el dinero involucrado, sino que el jugador afectado deja de confiar en el servidor y en varios casos termina abandonando la comunidad, lo que representa una pérdida directa de usuarios activos. Con el nuevo sistema, al exigir confirmación de ambas partes y garantizar que la transferencia se complete por completo o se cancele por completo, se evita ese desgaste y se protege la retención de los jugadores que ya invirtieron tiempo dentro de la economía del servidor.

Proceso de auditoría de tradeos: actualmente cada disputa que un administrador tiene que resolver le puede tomar horas de revisión manual entre registros dispersos, tiempo que deja de dedicarle a moderar el resto del servidor o a atender otras necesidades de la comunidad. Con un historial centralizado y ordenado cronológicamente de cada tradeo, ese costo en tiempo de moderación baja considerablemente, lo que le permite al equipo administrativo responder más rápido y sostener una comunidad más activa y conforme con la gestión del servidor.

Proceso de origen del dinero: como el dinero que circula en los tradeos sale originalmente de los empleos y de los comerciantes, si el pago de salarios se sigue manejando como una simple suma sin ningún control, el servidor pierde la capacidad de saber cuánto dinero nuevo está entrando realmente a la economía, lo que a la larga provoca hiperinflación y devalúa cada tradeo que se haga con ese dinero. Al registrar cada pago como una transacción formal emitida desde la cuenta del "Sistema", el servidor puede llevar un control real de cuánto dinero se

está inyectando y sostener una economía de tradeo estable que no pierda valor con el tiempo.

6. MODELO CONCEPTUAL DE LA SOLUCIÓN

6.1 Descripción de las entidades

- 1. Jugador:** Es el usuario del servidor de rol que participa en la economía, administra su inventario y puede comprar, vender o intercambiar propiedades y vehículos.
- 2. Cuenta:** Es el registro financiero asociado a un jugador o entidad que almacena el saldo de dinero disponible. Existe también una cuenta oficial del "Sistema".
- 3. Inventario/Items:** Son los artículos que los jugadores pueden poseer, adquirir, mejorar o revender dentro del juego.
- 4. Empleo:** Es la actividad laboral simulada que los jugadores realizan dentro del juego para ganar dinero.
- 5. Transacción:** Es el registro inmutable de cada movimiento económico realizado, ya sea el pago de un salario, una compra o un intercambio entre jugadores.
- 6. Servidor:** Representa la infraestructura técnica o entorno del sistema donde residen las configuraciones globales, jugadores y empleos.

6.2 Descripción de las relaciones

- **Posee (Jugador — Items):** Indica qué bienes o ítems están actualmente en posesión de un jugador específico.
- **Realiza (Jugador — Transaccion):** Vincula a un jugador con el registro de las transacciones financieras que ejecuta.
- **Tiene (Jugador — Cuenta):** Asocia al jugador con su cuenta bancaria correspondiente para almacenar su saldo.
- **Desempeña (Jugador — Empleo):** Conecta al jugador con el trabajo que realiza dentro del servidor de RP.
- **Aloja (Servidor — Jugador):** Indica que el servidor contiene o aloja los perfiles de los jugadores activos.

- **Definie (Servidor — Empleo):** Establece los puestos de trabajo y salarios parametrizados dentro del servidor.
- **Se hace (Items — Transaccion):** Especifica qué ítems están involucrados en una transacción comercial determinada.
- **Almacena (Cuenta — Items):** Relación que indica que la cuenta o inventario registra los ítems del usuario.
- **Actualiza saldos (Transaccion — Cuenta):** Pretende reflejar la acción de la transacción sobre el dinero en la cuenta.

6.3 Descripción de los atributos

- **Atributos de Jugador:** (*Id_Jugador, Credenciales*)

Id_Jugador: Identificador único usado para consultas o auditorías.

Credenciales: Datos de acceso que el sistema verifica al ingresar al servidor.

- **Atributos de Cuenta:** (*Id_Cuenta, Saldo inicial, Saldo disponible*)

Id_Cuenta: Identificador único, alfanumérico o numérico (ej. número de cuenta bancaria o ID de saldo), que permite diferenciar de forma inequívoca la cuenta bancaria de un jugador frente a las demás en la base de datos del servidor.

Saldo disponible: Cantidad de dinero actual que posee el jugador.

Saldo inicial: Monto asignado automáticamente la primera vez que el jugador entra al servidor.

- **Atributos de Item:** (*Id_Item, Nombre, Precio, Fecha, Restricción, Antigüedad*)

Id_Item : Identificador único del objeto activo.

Nombre/Tipo: Clasificación del bien (vehículo, propiedad, negocio, etc.).

Precio / Valor del ítem: Costo monetario del bien.

Fecha: Fecha en la que el propietario actual obtuvo el bien.

Restricción (Deuda / Cuota pendiente): Estado legal o financiero que impide que el bien sea transferido.

Uso / Antigüedad: Factor utilizado para calcular la depreciación y el valor de reventa.

- **Atributos de Empleo:** (*Id_Empleo, Nombre_Empleo, Tarifa_base, Horas_trabajadas*)

Id_Empleo : Código o identificador único asignado a cada tipo de trabajo dentro del servidor para gestionar de forma individual sus tarifas, salarios y permisos.

Nombre_Empleo: Denominación o título del trabajo.

Tarifa_base: Valor monetario que paga el empleo por unidad de trabajo.

Horas_trabajadas: Métrica de la jornada laboral realizada por el jugador.

- **Atributos de Transacción:** (*Id_Transaccion*, *Fecha_Hora*, *Monto*, *Usuarios_Involucrados*, *Bienes_afectados*)

Id_Transaccion: Identificador único generado tras una operación exitosa.

Fecha_Hora: Momento exacto en el que ocurre el movimiento.

Monto (Valor exacto abonado): Cantidad de dinero transferida.

Usuarios_Involucrados: Da precisión de la cuenta origen / Emisor que es quién envía el dinero o el bien y la cuenta destino que es quién recibe el dinero o el bien.

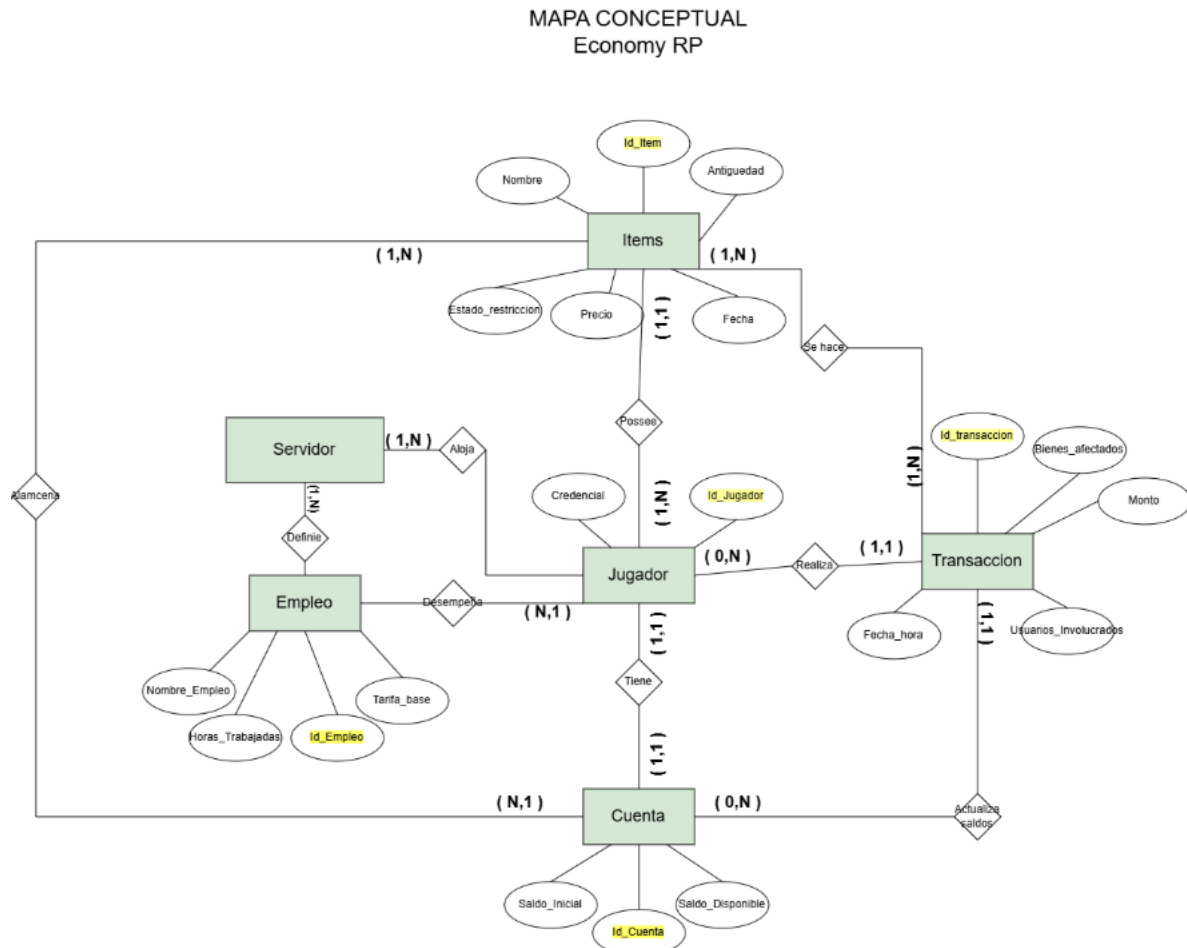
Bienes_afectados: El bien específico que cambió de propietario.

- **Atributos de Servidor:** (Ninguno que podamos identificar por el momento.)

6.4 Modelado conceptual: En este apartado hicimos varios modelos conceptuales los cuales fueron evolucionando gradualmente hasta llegar a un modelado conceptual adecuado.

0.1 Modelado piloto (Primera versión)

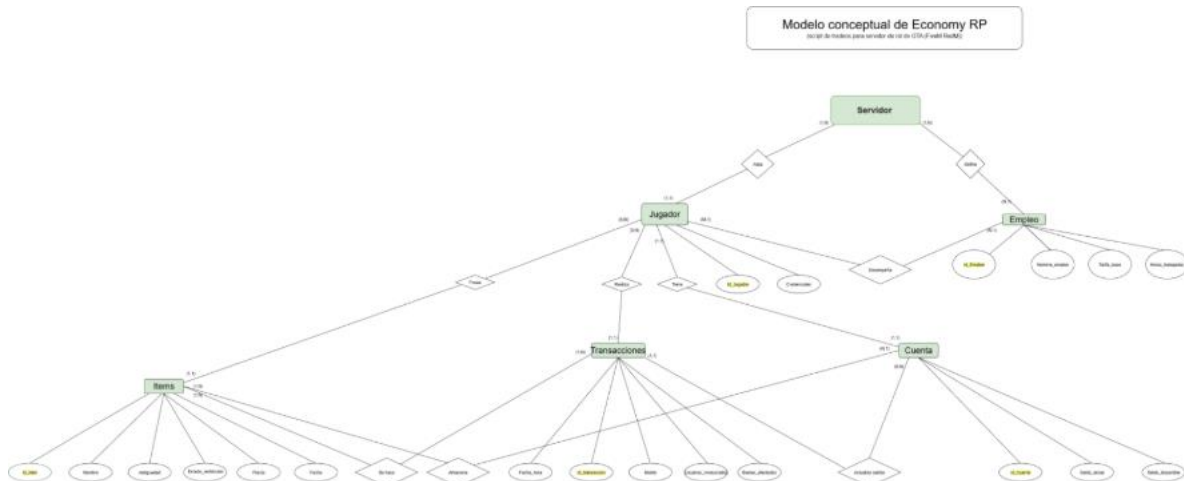
Primer modelado conceptual prototipo, estructuralmente muy desorganizado y de difícil lectura secuencial.



(Imagen PNG Modelo Conceptual Economy RP descargada de draw.io)

0.2 Modelado 2.0 (Orden cascada)

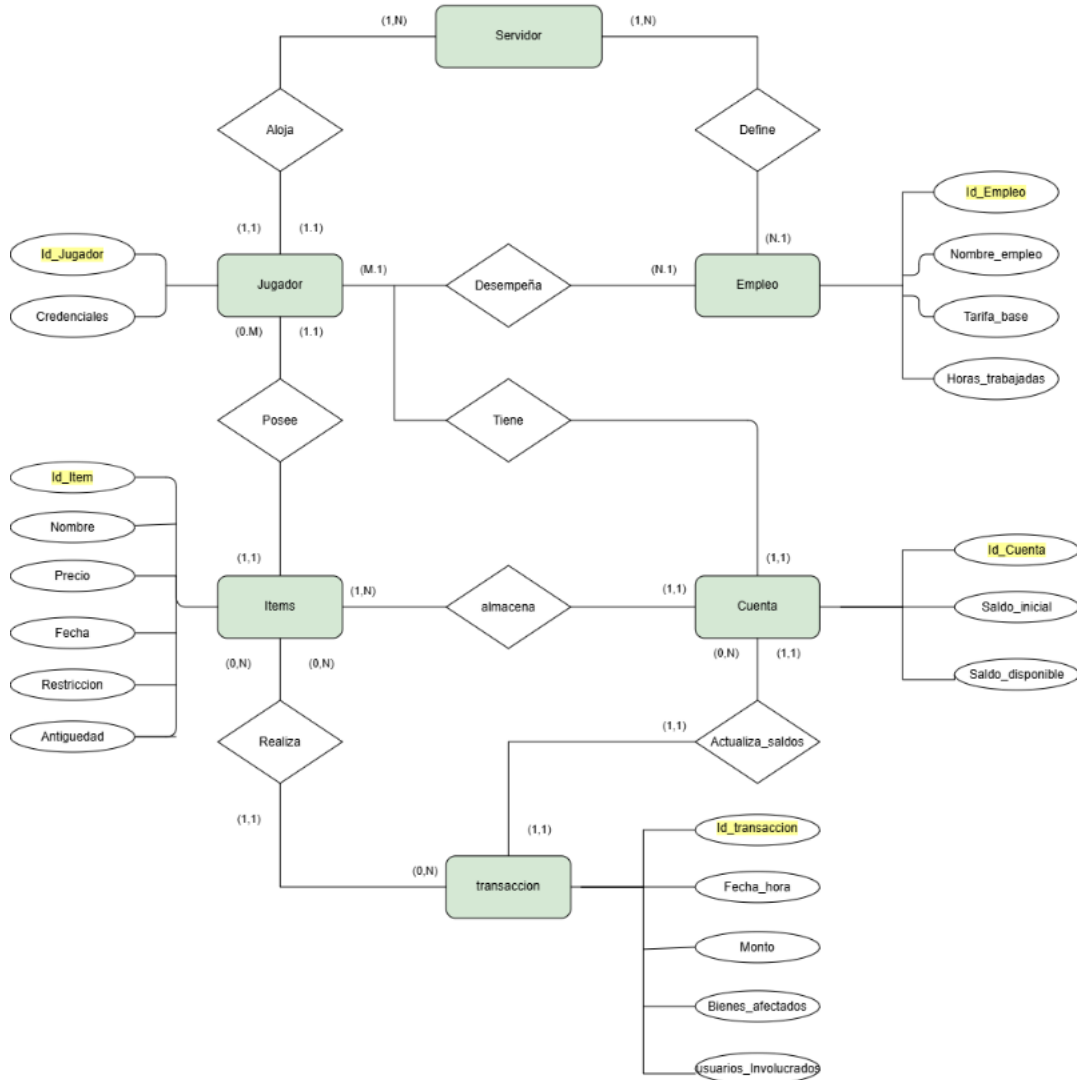
En esta fase del modelado optamos por una estructura secuencial (modo cascada) para que fuera visualmente más legible pero aun contaba con deficiencias organizacionales.



En nuestro modelado definitivo optamos por usar una estructura más lineal. La lectura aún conserva la medida secuencial y estéticamente los atributos están más cerca de las entidades. (Además a diferencia de los últimos modelados este obvia una relación ya que era parcialmente redundante)

Modelo conceptual de Economy RP

(script de tradeos para servidor de rol de GTA (FiveM RedM))



Enlaces a modelo conceptual Draw.io:

https://app.diagrams.net/#G1vccAkyamnuTZAjezohfrKgFDja_E1Yg4#%7B%22pageld%22%3A%22Dglu1eW9WFayZ3NLflwW%22%7D

https://drive.google.com/file/d/1vccAkyamnuTZAjezohfrKgFDja_E1Yg4/view?usp=s_haring

7. MODELO LÓGICO DE LA SOLUCIÓN

Para lograr los puntos 7.1, 7.2 y 7.3 primero se pasó el proyecto de aula y el modelo conceptual por el proceso de normalización para garantizar la integridad relacional de los datos y el cumplimiento de las reglas de negocio.

1FN: Eliminar atributos multivaluados o repetidos.

- **Atributos Atómicos:** Se eliminaron los atributos compuestos y multivaluados (como "Usuarios Involucrados" o "Bienes Afectados") identificados en las primeras fases conceptuales. En su lugar, se descompusieron las entidades y relaciones asociativas de forma que cada columna contenga un único valor indivisible de un dominio determinado.
- **Identificadores Únicos:** Cada entidad posee una Clave Primaria (PK) explícita que identifica de manera unívoca a cada registro en el sistema (ej. id_servidor, id_Jugador, id_Cuenta, id_Item, id_Transaccion...).

2FN: Eliminar dependencias parciales.

- **Cumplimiento de 1FN:** Todas las tablas se encuentran previamente en 1FN.
- **Dependencia Funcional Total:** Dado que todas las tablas del sistema utilizan claves primarias simples (de una sola columna), **no existen dependencias parciales**. Todos los atributos no clave de cada entidad dependen en su totalidad e individualmente de su clave primaria correspondiente. Por ejemplo, en T_Jornada Laboral, la fecha, las horas trabajadas y el monto pagado dependen explícitamente de la llave única id_Jornada.

3FN: Eliminar dependencias transitivas.

- **Cumplimiento de 2FN:** Todas las tablas están en 2FN.
- **Eliminación de Dependencias Transitivas y Valores Derivados:**
 - **Monto Histórico de Jornada:** En la tabla T_Jornada Laboral se reemplazó la columna Ganancia_Calculada por Monto_pagado. Esto evita calcular o derivar dinámicamente un valor a partir de Tarifa_Base (de T_Empleo) y Horas_Trabajadas, almacenándolo únicamente como un registro histórico inmutable para cumplir la 3FN.
 - **Cálculo de Antigüedad y Restricción:** En T_Item, el atributo Antigüedad se definió de tipo numérico (DECIMAL) para evitar almacenar marcas de tiempo duplicadas con Fecha, permitiendo calcular dinámicamente la depreciación sin generar dependencias transitivas dentro de la entidad.
 - **Atribución de Transacciones:** Las cuentas asociadas a las transacciones se separaron estructuralmente en la tabla T_Detalle_Transaccion, vinculando individualmente las llaves foráneas Cuenta_origen y Cuenta_destino a la tabla T_Cuenta, eliminando transitividad sobre la cabecera de la transacción.

7.1 Descripción de las tablas

T_Servidor: Representa la entidad o entorno técnico donde operan los distintos servidores de rol. Almacena la configuración general y la información global del servidor en la plataforma.

T_Jugador: Registra la información de los personajes o usuarios activos dentro de la economía del juego. Contiene sus credenciales de acceso y está vinculada directamente con el servidor al que pertenecen.

T_Empleo: Define el catálogo de trabajos y actividades laborales simuladas que ofrece un servidor. Especifica las tarifas base por hora que se utilizan para calcular el pago de salarios.

T_Jornada Laboral: Representa el registro individual de cada turno o jornada laboral completada por un jugador en un determinado trabajo. Sirve como constancia histórica del tiempo trabajado y del pago asignado.

T_Tipo_item : Catálogo de categorías de bienes (Vehículo, Propiedad, Negocio), evita repetir texto libre en cada ítem.

T_Item: Guarda los bienes, activos físicos o propiedades (vehículos, casas, negocios) que existen en el servidor. Mantiene el control sobre cuál jugador es el propietario actual, si el bien tiene alguna restricción o embargo activo y su estado de uso.

T_Cuenta: Registra las cuentas bancarias o monetarias asociadas a los jugadores para almacenar su saldo disponible. Permite también la existencia de cuentas del sistema para la emisión lícita de salarios o dinero base.

T_Negociacion_Tradeo : Registra el proceso de negociación entre dos jugadores antes de confirmarse, incluyendo el estado de confirmación de cada parte (RN010, RN012).

T_Transaccion: Bitácora inmutable que almacena los encabezados de cada movimiento monetario o comercial ejecutado en el sistema. Registra el monto total transferido, la marca de tiempo exacta y el estado de ejecución de la operación.

T_Detalle_Transaccion: Tabla de soporte que especifica los actores financieros involucrados en una transacción puntual, discriminando de forma precisa la cuenta emisora o de origen y la cuenta receptora o de destino.

7.2 Descripción de los atributos

Atributos de T_Servidor

- **id_servidor** : Clave primaria de la entidad. Identificador único y numérico asignado a cada servidor.
- **porcentaje_comision**: Porcentaje o tasa fija descontada automáticamente en cada transacción o tradeo para transferir al Sistema y frenar la hiperinflación del servidor (Regla RN006).
- **limite_bienes_por_jugador**: Número máximo de propiedades o vehículos que un solo jugador puede registrar a su nombre para evitar el acaparamiento de bienes (Regla RN007).
- **tiempo_enfriamiento_min**: Tiempo mínimo de espera (en minutos) que debe cumplir un jugador entre el fin de una jornada laboral y el inicio de la siguiente (Regla RN008).

Atributos de T_Jugador

- **id_Jugador**: Clave primaria. Identificador único de cada jugador o personaje.
- **Credenciales**: Cadena de caracteres que guarda las credenciales o tokens de autenticación del usuario.

- **Id_Servidor** : Clave foránea que referencia al servidor (T_Servidor) en el que está registrado el jugador.
- **id_jornada** : Clave foránea opcional que indica la jornada laboral activa que está desempeñando el personaje en tiempo real.
- **estado**: Estado actual de la cuenta del jugador en el servidor (ej. 'ACTIVO', 'BANCARROTA', 'SUSPENDIDO') (Regla RN009).

Atributos de T_Empleo

- **id_Empleo** : Clave primaria. Código único asignado a cada tipo de trabajo disponible.
- **Nombre_Empleo**: Denominación o título público del trabajo (ej. Minero, Policía, Mecánico).
- **Tarifa_base**: Valor monetario asignado por hora o unidad de trabajo completada.
- **Id_Servidor**: Clave foránea que asocia el empleo con el servidor que lo parametrizó (T_Servidor).

Atributos de T_Jornada Laboral

- **id_Jornada**: Clave primaria. Identificador único del registro de la jornada realizada.
- **Id_Jugador**: Clave foránea que vincula al jugador (T_Jugador) que desempeñó el trabajo.
- **Id_Empleo**: Clave foránea que identifica el trabajo (T_Empleo) realizado en la jornada.
- **Horas_Trabajadas**: Cantidad de tiempo o unidades laborales invertidas durante el turno.
- **Fecha_Hora**: Fecha y hora exacta en la que finalizó el turno laboral.
- **Monto_pagado**: Cantidad exacta de dinero acreditada al jugador por esa jornada específica.

Atributos de T_Item

- **id_Item**: Clave primaria. Identificador único del bien (número de chasis, placa, matrícula o ID de catastro).
- **Nombre**: Nombre o modelo comercial del bien.
- **Precio**: Valor monetario o costo base comercial del objeto.
- **Fecha**: Fecha y hora en la que el propietario actual adquirió el activo.
- **Restriccion**: Estado legal del bien (ej. 'Ninguna', 'Embargado', 'Deuda Pendiente').

- **antiguedad_dias**: Métrica exacta en días de uso acumulados del bien, utilizada para aplicar fórmulas de depreciación y valor de reventa.
- **Id_jugador**: Clave foránea que indica qué jugador (T_Jugador) es el dueño único del ítem.

Atributos de T_Negociacion_Tradeo

- **id_negociacion** : Clave primaria. Identificador único de la mesa de negociación o propuesta de tradeo activa.
- **id_jugador_1**: Clave foránea que identifica al jugador emisor o que inicia la propuesta de intercambio.
- **id_jugador_2**: Clave foránea que identifica al jugador receptor o contraparte en la negociación.
- **id_item_ofrecido**: Clave foránea hacia T_Item. Representa el bien o activo específico que se está ofreciendo en el trato.
- **monto_ofrecido**: Cantidad de dinero ofrecida como parte de la permuta o compra directa.
- **confirmacion_j1**: Indicador booleano (verdadero/falso) que valida si el jugador 1 aceptó los términos finales del intercambio (Regla RN010).
- **confirmacion_j2**: Indicador booleano (verdadero/falso) que valida si el jugador 2 aceptó los términos finales del intercambio (Regla RN010).
- **estado**: Estado del proceso de negociación (ej. 'PENDIENTE', 'CONFIRMADO', 'CANCELADO').
- **fecha_creacion**: Marca de tiempo que registra el momento exacto en que se abrió la propuesta de tradeo.

Atributos de T_Cuenta

- **id_Cuenta**: Clave primaria. Identificador único de la cuenta bancaria o monedero.
- **Saldo_inicial**: Monto inicial con el que fue apertura la cuenta.
- **Saldo_disponible**: Balance o dinero neto disponible en la cuenta del usuario.
- **id_jugador**: Clave foránea que relaciona la cuenta con su jugador titular (T_Jugador). Puede ser nula en caso de representar la cuenta del Sistema del Servidor (RN005).
- **tipo_cuenta**: Clasificación de la cuenta dentro del sistema (ej. 'PERSONAL', 'SISTEMA_EMITOR') para diferenciar las cuentas de los jugadores de la cuenta oficial del servidor (Regla RN005).

Atributos de T_Transaccion

- **id_Transaccion:** Clave primaria. Identificador único e inmutable del movimiento monetario.
- **Fecha_hora:** Marca temporal exacta (año, mes, día, hora) en que se ejecutó la transacción.
- **Monto:** Cantidad total de dinero transferida en la operación.
- **Item_id_afectado:** Clave foránea hacia T_Item. Opcional, presente únicamente cuando la transacción implica la transferencia de propiedad de un bien.
- **Estado_transaccion :** Estado de ejecución del movimiento (ej. 'COMPLETADA', 'CANCELADA', 'REVERTIDA').
- **id_negociacion** (*INT, NULLABLE*): Clave foránea que vincula la transacción final con la mesa de negociación (T_Negociacion_Tradeo) que la originó.
- **id_jornada:** Clave foránea hacia T_Jornada_Laboral, presente cuando la transacción corresponde al pago formal de un salario.
- **tipo_transaccion:** Categorización del movimiento económico (ej. 'PAGO_SALARIO', 'TRADEO_P2P', 'COMPRA_COMERCIANTE').

Atributos de T_Detalle_Transaccion

- **id_detalle:** Clave primaria. Identificador único del registro de detalle de la transferencia.
- **Id_transaccion:** Clave foránea que asocia este detalle con su transacción encabezado (T_Transaccion).
- **Cuenta_origen:** Clave foránea que indica la cuenta (T_Cuenta) de la cual se debitó el dinero.
- **Cuenta_destino:** Clave foránea que indica la cuenta (T_Cuenta) a la cual se le acreditó el monto.
- **monto_detalle:** Valor monetario específico discriminado en este asiento o movimiento secundario.
- **concepto:** Descripción o justificación del movimiento (ej. 'PAGADO_A_VENDEDOR', 'COMISION_DRENAJE_SISTEMA') (Regla RN006).

7.3 Modelado lógico (Razonamiento)

En este script escrito en lenguaje DBML se diseñó y estructuró el modelo lógico relacional para el sistema económico del servidor de rol, organizando las tablas en módulos funcionales diferenciados por color (headercolor) y agrupaciones visuales

(TableGroup) con el objetivo de erradicar la duplicación de objetos, las estafas en intercambios P2P y la falta de auditoría en el juego. En primer lugar, dentro de la capa de administración, se definió la tabla de servidores (T_Servidor) para almacenar parámetros globales de control como comisiones, límites de posesiones por usuario y tiempos de espera, junto con la tabla de empleos (T_Empleo) que establece el catálogo de trabajos y sus salarios base. En el módulo financiero, se representó a los usuarios (T_Jugador) vinculándolos a su balance bancario (T_Cuenta) mediante una relación explícita de uno a uno (Ref: T_Cuenta.id_jugador - T_Jugador.id_jugador), la cual admite la ausencia de un jugador para identificar la cuenta oficial del Sistema destinada a la inyección controlada de dinero. En la capa operativa y comercial, se incorporó la tabla de inventario (T_Item) para rastrear la propiedad de activos, su desgaste y posibles embargos o deudas, la tabla de registro de trabajo (T_Jornada_Laboral) para almacenar los turnos completados e historial de pagos evitando la generación imprevista de dinero ficticio, y la tabla de intercambios (T_Negociacion_Tradeo) encargada de procesar ofertas entre dos usuarios (id_jugador_1 e id_jugador_2) exigiendo una doble validación booleana (confirmacion_j1 y confirmacion_j2) antes de consolidar la transferencia de activos. Por último, en el módulo contable de trazabilidad, se implementaron las tablas de encabezado de movimientos (T_Transaccion) y desglose de movimientos (T_Detalle_Transaccion), las cuales aíslan las cuentas involucradas registrando por separado el débito (cuenta_origen) y el crédito (cuenta_destino), eliminando así celdas con datos repetidos o múltiples usuarios para cumplir con la primera forma normal (1FN) y proporcionar un registro inalterable para la resolución de disputas.

7.4 Modelado lógico

_Script lenguaje DBML:

```
Table T_Servidor {  
  id_servidor int [pk] // PK  
  nombre varchar  
  porcentaje_comision decimal  
  limite_bienes_por_jugador int  
  tiempo_enfriamiento_min int  
}
```

```
Table T_Jugador {  
  id_jugador int [pk] // PK  
  id_servidor int [ref: > T_Servidor.id_servidor] // FK hacia T_Servidor  
  credenciales varchar  
  estado varchar  
}
```

```
Table T_Empleo {  
  id_empleo int [pk] // PK  
  id_servidor int [ref: > T_Servidor.id_servidor] // FK hacia T_Servidor  
  nombre_empleo varchar  
  tarifa_base decimal  
}
```

```
Table T_Jornada_Laboral {  
  id_jornada int [pk] // PK  
  id_jugador int [ref: > T_Jugador.id_jugador] // FK hacia T_Jugador
```

```
id_empleo int [ref: > T_Empleo.id_empleo] // FK hacia T_Empleo
horas_trabajadas decimal
fecha_hora timestamp
ganancia_calculada decimal
}
```

```
Table T_Cuenta {
  id_cuenta int [pk] // PK
  id_jugador int [unique, ref: > T_Jugador.id_jugador] // FK hacia T_Jugador (relacion
1 a 1)
  tipo_cuenta varchar
  saldo_inicial decimal
  saldo_disponible decimal
}
```

```
Table T_Item {
  id_item int [pk] // PK
  id_jugador int [ref: > T_Jugador.id_jugador] // FK hacia T_Jugador
  nombre varchar
  precio decimal
  fecha timestamp
  restriccion varchar
  antiguedad_dias int
}
```

```
Table T_Negociacion_Tradeo {
  id_negociacion int [pk] // PK
  id_jugador_1 int [ref: > T_Jugador.id_jugador] // FK hacia T_Jugador (propone)
  id_jugador_2 int [ref: > T_Jugador.id_jugador] // FK hacia T_Jugador (recibe)
```

```
id_item_ofrecido int [ref: > T_Item.id_item] // FK hacia T_Item
monto_ofrecido decimal
confirmacion_j1 boolean
confirmacion_j2 boolean
estado varchar
fecha_creacion timestamp
}
```

```
Table T_Transaccion {
  id_transaccion int [pk] // PK
  id_item_afectado int [ref: > T_Item.id_item] // FK hacia T_Item
  id_negociacion int [ref: > T_Negociacion_Tradeo.id_negociacion // FK hacia
T_Negociacion_Tradeo
  id_jornada int [ref: > T_Jornada_Laboral.id_jornada] // FK hacia
T_Jornada_Laboral
  tipo_transaccion varchar
  estado_transaccion varchar
  monto decimal
  fecha_hora timestamp
}
```

```
Table T_Detalle_Transaccion {
  id_detalle int [pk] // PK
  id_transaccion int [ref: > T_Transaccion.id_transaccion] // FK hacia T_Transaccion
  cuenta_origen int [ref: > T_Cuenta.id_cuenta] // FK hacia T_Cuenta
  cuenta_destino int [ref: > T_Cuenta.id_cuenta] // FK hacia T_Cuenta
  monto_detalle decimal
  concepto varchar
}
```

[illegible]

30